

Week - 10
Adarsh Ranjan
230962278, AIMA - 52

Lab Exercises:

1. Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

```
#include <stdio.h>
#include <cuda_runtime.h>

#define N 4
#define BLOCK_SIZE 2

__global__ void matrixMul(int *A, int *B, int *C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < width && col < width) {
        int sum = 0;
        for (int k = 0; k < width; k++) {
            sum += A[row * width + k] * B[k * width + col];
        }
        C[row * width + col] = sum;
    }
}

int main() {
    int h_A[N][N], h_B[N][N], h_C[N][N];
    int *d_A, *d_B, *d_C;
    int size = N * N * sizeof(int);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            h_A[i][j] = i + j;
            h_B[i][j] = i * j;
        }
    }
}
```

```

cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

dim3 threads(BLOCK_SIZE, BLOCK_SIZE);
dim3 blocks((N + BLOCK_SIZE - 1) / BLOCK_SIZE, (N + BLOCK_SIZE - 1) /
BLOCK_SIZE);

matrixMul<<<blocks, threads>>>(d_A, d_B, d_C, N);

cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
printf("Matrix A:\n");

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", h_A[i][j]);
    }
    printf("\n");
}
printf("Matrix B:\n");

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", h_B[i][j]);
    }
    printf("\n");
}
printf("Matrix C:\n");

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", h_C[i][j]);
    }
    printf("\n");
}

printf("Adarsh Ranjan, 230962278");

```

```

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}

```

```

(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ nvcc q1.cu -o q1
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ ./q1
Matrix A:
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
Matrix B:
0 0 0 0
0 1 2 3
0 2 4 6
0 3 6 9
Matrix C:
0 14 28 42
0 20 40 60
0 26 52 78
0 32 64 96
Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$

```

2. Write a program in CUDA to improve the performance of 1D parallel convolution using constant Memory.

```

#include <stdio.h>
#include <cuda_runtime.h>

#define N 8
#define MASK_WIDTH 3

__constant__ int M[MASK_WIDTH];

__global__ void convolution1DConstant(int *N_arr, int *P, int width) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int radius = MASK_WIDTH / 2;

    if (i < width) {
        int sum = 0;

```

```

        for (int j = 0; j < MASK_WIDTH; j++) {
            int idx = i - radius + j;
            if (idx >= 0 && idx < width) {
                sum += N_arr[idx] * M[j];
            }
        }
        P[i] = sum;
    }
}

int main() {
    int h_N[N] = {1,2,3,4,5,6,7,8};
    int h_M[MASK_WIDTH] = {1,0,-1};
    int h_P[N];

    int *d_N, *d_P;

    cudaMalloc((void**)&d_N, N * sizeof(int));
    cudaMalloc((void**)&d_P, N * sizeof(int));

    cudaMemcpy(d_N, h_N, N * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpyToSymbol(M, h_M, MASK_WIDTH * sizeof(int));

    convolution1DConstant<<<1, N>>>(d_N, d_P, N);

    cudaMemcpy(h_P, d_P, N * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < N; i++) {
        printf("%d ", h_P[i]);
    }
    printf("\n");
    printf("Adarsh Ranjan, 230962278");

    cudaFree(d_N);
    cudaFree(d_P);

    return 0;
}

```

```
● (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ nvcc q2.cu -o q2
● (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ ./q2
-2 -2 -2 -2 -2 -2 -2 7
○ Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$
```

3. Write a program in CUDA to perform tiled 1D convolution operation on the input array N of size width using the mask array, M of size mask_width, to produce the resultant array P of size width.

```
#include <stdio.h>
#include <cuda_runtime.h>

#define WIDTH 8
#define MASK_WIDTH 3
#define TILE_SIZE 4

__constant__ int M[MASK_WIDTH];

__global__ void tiledConvolution1D(int *N, int *P, int width) {
    __shared__ int N_s[TILE_SIZE + MASK_WIDTH - 1];

    int tx = threadIdx.x;
    int start = blockIdx.x * TILE_SIZE;
    int radius = MASK_WIDTH / 2;
    int gIndex = start + tx;

    if (gIndex < width)
        N_s[tx + radius] = N[gIndex];
    else
        N_s[tx + radius] = 0;

    if (tx < radius) {
        int left = gIndex - radius;
        N_s[tx] = (left >= 0) ? N[left] : 0;
    }

    if (tx < radius) {
        int right = gIndex + TILE_SIZE;
        N_s[tx + TILE_SIZE + radius] = (right < width) ? N[right] : 0;
    }
}
```

```

    }

    __syncthreads();

    if (gIndex < width) {
        int sum = 0;
        for (int j = 0; j < MASK_WIDTH; j++) {
            sum += N_s[tx + j] * M[j];
        }
        P[gIndex] = sum;
    }
}

int main() {
    int h_N[WIDTH] = {1,2,3,4,5,6,7,8};
    int h_M[MASK_WIDTH] = {1,0,-1};
    int h_P[WIDTH];

    int *d_N, *d_P;

    cudaMalloc((void**)&d_N, WIDTH * sizeof(int));
    cudaMalloc((void**)&d_P, WIDTH * sizeof(int));

    cudaMemcpy(d_N, h_N, WIDTH * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpyToSymbol(M, h_M, MASK_WIDTH * sizeof(int));

    int blocks = (WIDTH + TILE_SIZE - 1) / TILE_SIZE;

    tiledConvolution1D<<<blocks, TILE_SIZE>>>(d_N, d_P, WIDTH);

    cudaMemcpy(h_P, d_P, WIDTH * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < WIDTH; i++) {
        printf("%d ", h_P[i]);
    }
    printf("\n");
    printf("Adarsh Ranjan, 230962278");

    cudaFree(d_N);
    cudaFree(d_P);
}

```

```
    return 0;  
}
```

```
• (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ nvcc q3.cu -o q3  
• (base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$ ./q3  
-2 -2 -2 -2 -2 -2 -2 7  
○ Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week10$
```